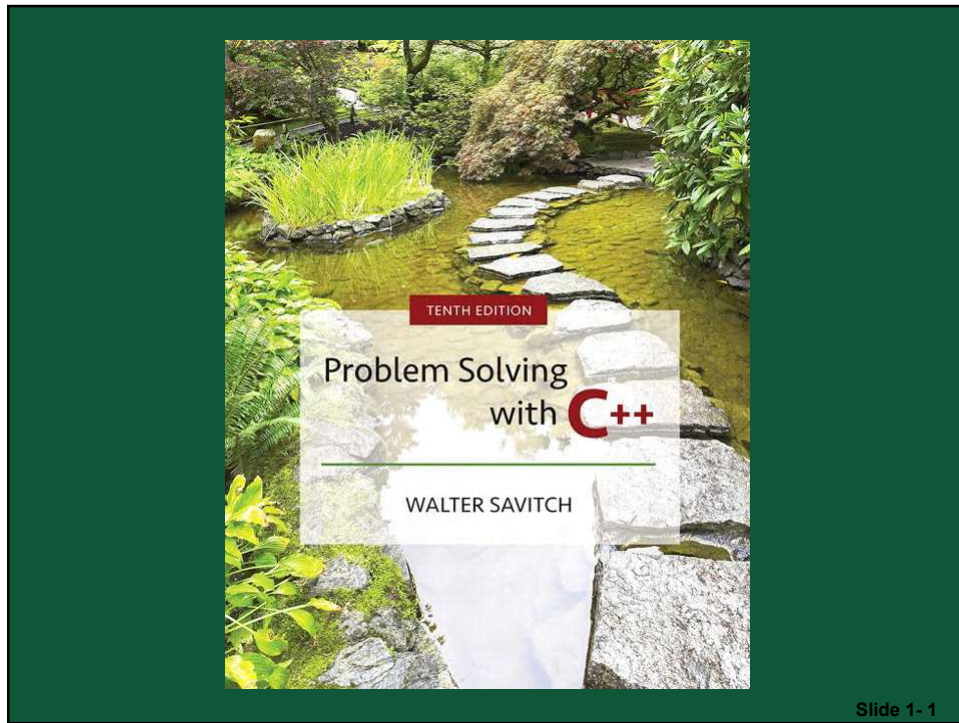




1

Chapter 8

Strings and Vectors

PEARSON

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

2

Overview

8.1 An Array Type for Strings

8.2 The Standard **string** Class

8.3 Vectors

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 3

3

8.1

An Array Type for Strings



Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

4

An Array Type for Strings

- C-strings can be used to represent strings of characters
 - C-strings are stored as arrays of characters
 - C-strings use the null character '\0' to end a string
 - The Null character is a single character
 - To declare a C-string variable, declare an array of characters:

```
char s[11];
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 5

5

C-string Details

- Declaring a C-string as `char s[10]` creates space for only nine characters
 - The null character terminator requires one space
- A C-string variable does not need a size variable
 - The null character immediately follows the last character of the string
- Example

s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[7]	s[8]	s[9]
H	i		M	o	m	!	\0	?	?

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 6

6

C-string Declaration

- To declare a C-string variable, use the syntax:

```
char Array_name[ Maximum_cString_Size + 1];
```

- + 1 reserves the additional character needed by '\0'

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 7

7

Initializing a C-string

- To initialize a C-string during declaration:

```
char myMessage[20] = "Hi there.";
```

- The null character '\0' is added for you

- Another alternative:

```
char shortString[ ] = "abc";
```

but not this:

```
char shortString[ ] = {'a', 'b', 'c'};
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 8

8

C-string error

- This attempt to initialize a C-string does not cause the `\0` to be inserted in the array

```
char shortString[ ] = {'a', 'b', 'c'};
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 9

9

Don't Change `\0`

- Do not to replace the null character when manipulating indexed variables in a C-string

- If the null character is lost, the array cannot act like a C-string, Example:

```
int index = 0;
while (ourString[index] != '\0')
{
    ourString[index] = 'X';
    index++;
}
```

- This code depends on finding the null character!

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 10

10

Safer Processing of C-strings

- The loop on the previous slide depended on finding the '\0' character
 - It would be wiser to use this version in case the '\0' character had been removed

```
int index = 0;
while (ourString[index] != '\0'
      && index < SIZE)
{
    ourString[index] = 'X';
    index++;
}
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 11

11

Assignment With C-strings

- This statement is illegal:

```
aString = "Hello";
```

- This is an assignment statement, not an initialization
- The assignment operator does not work with C-strings

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 12

12

Assignment of C-strings

- A common method to assign a value to a C-string variable is to use `strcpy`, defined in the `cstring` library. Example:

```
#include <cstring>
...
char aString [11];
strcpy (aString, "Hello");
```

Places "Hello" followed by the null character in `aString`

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 13

13

A Problem With `strcpy`

- `strcpy` can create problems if not used carefully
 - `strcpy` does not check the declared length of the first argument
 - It is possible for `strcpy` to write characters beyond the declared size of the array

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 14

14

A Solution for strcpy

- Many versions of C++ have a safer version of `strcpy` named `strncpy`
 - `strncpy` uses a third argument representing the maximum number of characters to copy.

Example:

```
char anotherString[10];  
strncpy(anotherString, aStringVariable, 9);
```

This code copies up to 9 characters into `anotherString`, leaving one space for `'\0'`

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 15

15

== Alternative for C-strings

- The `==` operator does not work as expected with C-strings
 - The predefined function `strcmp` is used to compare C-string variables. Example:

```
#include <cstring>  
...  
if (strcmp(cString1, cString2))  
    cout << "Strings are not the same.";  
else  
    cout << "String are the same.";
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 16

16

strcmp's logic

- `strcmp` compares the numeric codes of elements in the C-strings a character at a time
 - If the two C-strings are the same, `strcmp` returns 0
 - 0 is interpreted as false
 - As soon as the characters do not match
 - `strcmp` returns a negative value if the numeric code in the first parameter is less
 - `strcmp` returns a positive value if the numeric code in the second parameter is less
 - Non-zero values are interpreted as true

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 17

17

More C-string Functions

- The `cstring` library includes other functions
 - `strlen` returns the number of characters in a string

```
int x = strlen(aString);
```
 - `strcat` concatenates two C-strings
 - The 2nd argument is added to the end of the 1st
 - The result is placed in the first argument
 - Example:

```
char stringVar[20] = "The rain";
strcat(stringVar, "in Spain");
```

Now `stringVar` contains "The rainin Spain"

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 18

18

The strncat Function

- `strncat` is a safer version of `strcat`
 - A third parameter specifies a limit for the number of characters to concatenate
 - Example:

```
char stringVar[20] = "The rain";  
strncat(stringVar, "in Spain", 11);
```

Display 8.1 (1)

Display 8.1 (2)

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 19

19

Display 8.1 (1/2)

Back

Next

Some Predefined C-String Functions in `<cstring>` (part 1 of 2)

Function	Description	Cautions
<code>strcpy(Target_String_Var, Src_String)</code>	Copies the C-string value <i>Src_String</i> into the C-string variable <i>Target_String_Var</i> .	Does not check to make sure <i>Target_String_Var</i> is large enough to hold the value <i>Src_String</i> .
<code>strncpy(Target_String_Var, Src_String, Limit)</code>	The same as the two-argument <code>strcpy</code> except that at most <i>Limit</i> characters are copied.	If <i>Limit</i> is chosen carefully, this is safer than the two-argument version of <code>strcpy</code> . Not implemented in all versions of C++.
<code>strcat(Target_String_Var, Src_String)</code>	Concatenates the C-string value <i>Src_String</i> onto the end of the C string in the C-string variable <i>Target_String_Var</i> .	Does not check to see that <i>Target_String_Var</i> is large enough to hold the result of the concatenation.

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 20

20

Display 8.1 (2/2)

[Back](#)[Next](#)

Some Predefined C-String Functions in <cstring> (part 2 of 2)

<code>strncat(<i>Target_String_Var</i>, <i>Src_String</i>, <i>Limit</i>)</code>	The same as the two-argument <code>strcat</code> except that at most <i>Limit</i> characters are appended.	If <i>Limit</i> is chosen carefully, this is safer than the two-argument version of <code>strcat</code> . Not implemented in all versions of C++.
<code>strlen(<i>Src_String</i>)</code>	Returns an integer equal to the length of <i>Src_String</i> . (The null character, <code>'\0'</code> , is not counted in the length.)	
<code>strcmp(<i>String_1</i>, <i>String_2</i>)</code>	Returns 0 if <i>String_1</i> and <i>String_2</i> are the same. Returns a value < 0 if <i>String_1</i> is less than <i>String_2</i> . Returns a value > 0 if <i>String_1</i> is greater than <i>String_2</i> (that is, returns a nonzero value if <i>String_1</i> and <i>String_2</i> are different). The order is lexicographic.	If <i>String_1</i> equals <i>String_2</i> , this function returns 0, which converts to <code>false</code> . Note that this is the reverse of what you might expect it to return when the strings are equal.
<code>strncmp(<i>String_1</i>, <i>String_2</i>, <i>Limit</i>)</code>	The same as the two-argument <code>strcat</code> except that at most <i>Limit</i> characters are compared.	If <i>Limit</i> is chosen carefully, this is safer than the two-argument version of <code>strcmp</code> . Not implemented in all versions of C++.

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 21

21

C-strings as Arguments and Parameters

- C-string variables are arrays
- C-string arguments and parameters are used just like arrays
 - If a function changes the value of a C-string parameter, it is best to include a parameter for the declared size of the C-string
 - If a function does not change the value of a C-string parameter, the null character can detect the end of the string and no size argument is needed

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 22

22

C-string Output

- C-strings can be output with the insertion operator

- Example:

```
char news[ ] = "C-strings";  
cout << news << " Wow." << endl;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 23

23

C-string Input

- The extraction operator >> can fill a C-string

- Whitespace ends reading of data

- Example:

```
char a[80], b[80];  
cout << "Enter input: " << endl;  
cin >> a >> b;  
cout << a << b << "End of Output";
```

could produce:

Enter input:

Do be do to you!

DobeEnd of Output

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 24

24

Reading an Entire Line

- Predefined member function `getline` can read an entire line, including spaces
 - `getline` is a member of all input streams
 - `getline` has two arguments
 - The first is a C-string variable to receive input
 - The second is an integer, usually the size of the first argument specifying the maximum number of elements in the first argument `getline` is allowed to fill

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 25

25

Using `getline`

- The following code is used to read an entire line including spaces into a single C-string variable

```
char a[80];  
cout << "Enter input:\n";  
cin.getline(a, 80);  
cout << a << End Of Output\n";
```

and could produce:

```
Enter some input:  
Do be do to you!  
Do be do to you!End of Output
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 26

26

getline wrap up

- `getline` stops reading when the number of characters, less one, specified in the second argument have been placed in the C-string
 - one character is reserved for the null character
 - `getline` stops even if the end of the line has not been reached

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 27

27

getline and Files

- C-string input and output work the same way with file streams
 - Replace `cin` with the name of an input-file stream

```
inStream >> cString;
inStream.getline(cString, 80);
```

Replace `cout` with the name of an output-file stream

```
out_stream << cString;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 28

28

getline syntax

- Syntax for using `getline` is

```
cin.getline(String_Var, Max_Characters + 1);
```

- `cin` can be replaced by any input stream
- `Max_Characters + 1` reserves one element for the null character

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 29

29

C-String to Numbers

- "1234" is a string of characters
- 1234 is a number
- When doing numeric input, it is useful to read input as a string of characters, then convert the string to a number
 - Reading money may involve a dollar sign
 - Reading percentages may involve a percent sign

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 30

30

C-strings to Integers

- To read an integer as characters
 - Read input as characters into a C-string, removing unwanted characters
 - Use the predefined function `atoi` to convert the C-string to an int value

Example:

`atoi("1234")` returns the integer 1234
`atoi("#123")` returns 0 : # is not a digit

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 31

31

C-string to long

- Larger integers can be converted using the predefined function `atol`
 - `atol` returns a value of type long

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 32

32

C-string to double

- C-strings can be converted to type double using the predefined function `atof`
- `atof` returns a value of type double
 - Example:
`atof("9.99")` returns 9.99
`atof("$9.99")` returns 0.0 because the \$ is not a digit

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 33

33

Library `cstdlib`

- The conversion functions
`atoi`
`atol`
`atof`
are found in the library `cstdlib`
- To use the functions use the include directive

```
#include <cstdlib>
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 34

34

Numeric Input

- We now know how to convert C-strings to numbers
- How do we read the input?
 - Function `readAndClean`, in Display 8.2...
 - Reads a line of input
 - Discards all characters other than the digits '0' through '9'
 - Uses `atoi` to convert the "cleaned-up" C-string to `int`

Display 8.2 (1)

Display 8.2 (2)

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 35

35

Display 8.2 (1/2)



```
1 //Demonstrates the function readAndClean.
2 #include <iostream>
3 #include <cstdlib>
4 #include <cctype>
5
6 void readAndClean(int& n);
7 //Reads a line of input. Discards all symbols except the digits. Converts
8 //the C string to an integer and sets n equal to the value of this integer.
9
10 void newLine( );
11 //Discards all the input remaining on the current input line.
12 //Also discards the '\n' at the end of the line.
13
14 int main( )
15 {
16     using namespace std;
17     int n;
18     char ans;
19     do
20     {
21         cout << "Enter an integer and press Return: ";
22         readAndClean(n);
23         cout << "That string converts to the integer " << n << endl;
24         cout << "Again? (yes/no): ";
25         cin >> ans;
26         newLine( );
27     } while ( (ans != 'n') && (ans != 'N') );
28     return 0;
29 }
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 36

36

```

30 //Uses iostream, cstdlib, and ctype:
31 void readAndClean(int& n)
32 {
33     using namespace std;
34     const int ARRAY_SIZE = 6;
35     char digitString[ARRAY_SIZE];
36
37     char next;
38     cin.get(next);
39     int index = 0;
40     while (next != '\n')
41     {
42         if ((isdigit(next)) && (index < ARRAY_SIZE - 1))
43         {
44             digitString[index] = next;
45             index++;
46         }
47         cin.get(next);
48     }
49     digitString[index] = '\0';
50     n = atoi(digitString);
51 }
52 //Uses iostream:
53 void newLine( )
54 {
55     using namespace std;
56     <The rest of the definition of newLine is given in Display 6.7.>

```

Sample Dialogue

```

Enter an integer and press Return:  $ 100
That string converts to the integer 100
Again? (yes/no):  yes
Enter an integer and press Return:  100
That string converts to the integer 100
Again? (yes/no):  yes
Enter an integer and press Return:  99%
That string converts to the integer 99
Again? (yes/no):  yes
Enter an integer and press Return:  23% &&5 *12
That string converts to the integer 23512
Again? (yes/no):  no

```

Display 8.2 (2/2)

← Back
Next →

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.
Slide 8- 37

37

Confirming Input

- Function `get_int`, from Display 8.3...
 - Uses `readAndClean` to read the user's input
 - Allows the user to reenter the input until the user is satisfied with the number computed from the input string

Display 8.3 (1)

Display 8.3 (2)

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.
Slide 8- 38

38

Display 8.3 (1/3)



DISPLAY 8.3 Robust Input Function

```
1 //Demonstration program for improved version of getInt.
2 #include <iostream>
3 #include <cstdlib>
4 #include <cctype>

5 void readAndClean(int& n);
6 //Reads a line of input. Discards all symbols except the digits. Converts
7 //the C string to an integer and sets n equal to the value of this integer.

8 void newLine( );
9 //Discards all the input remaining on the current input line.
10 //Also discards the '\n' at the end of the line.

11 void getInt(int& inputNumber);
12 //Gives inputNumber a value that the user approves of.
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 39

39

```
13 int main( )
14 {
15     using namespace std;
16     int inputNumber;
17     getInt(inputNumber);
18     cout << "Final value read in = " << inputNumber << endl;
19     return 0;
20 }

21 //Uses iostream and readAndClean:
22 void getInt(int& inputNumber)
23 {
24     using namespace std;
25     char ans;
26     do
27     {
28         cout << "Enter input number: ";
29         readAndClean(inputNumber);
30         cout << "You entered " << inputNumber
31             << " Is that correct? (yes/no): ";
32         cin >> ans;
33         newLine( );
34     } while ((ans != 'y') && (ans != 'Y'));
35 }

36 //Uses iostream, cstdlib, and ctype:
37 void readAndClean(int& n)

<The rest of the definition of readAndClean is given in Display 8.2.>

38 //Uses iostream:
39 void newLine( )

<The rest of the definition of newLine is given in Display 8.2.>
```

Display 8.3 (2/3)

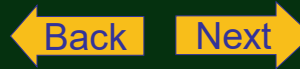


Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 40

40

Display 8.3 (3/3)



Sample Dialogue

```
Enter input number: $57
You entered 57 Is that correct? (yes/no): no
Enter input number: $77*5xa
You entered 775 Is that correct? (yes/no): no
Enter input number: 77
You entered 77 Is that correct? (yes/no): no
Enter input number: $75
You entered 75 Is that correct? (yes/no): yes
Final value read in = 75
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 41

41

Section 8.1 Conclusion

- Can you
 - Describe the benefits of reading numeric data as characters before converting the characters to a number?
 - Write code to do input and output with C-strings?
 - Use the atoi, atol, and atof functions?
 - Identify the character that ends a C-string?

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 42

42

8.2

The Standard **string** Class



Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

43

The Standard string Class

- The string class allows the programmer to treat strings as a basic data type
 - No need to deal with the implementation as with C-strings
- The string class is defined in the string library and the names are in the standard namespace
 - To use the string class you need these lines:

```
#include <string>  
using namespace std;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 44

44

Assignment of Strings

- Variables of type string can be assigned with the = operator
 - Example:

```
string s1, s2, s3;  
...  
s3 = s2;
```
- Quoted strings are type cast to type string
 - Example:

```
string s1 = "Hello Mom!";
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 45

45

Using + With strings

- Variables of type string can be concatenated with the + operator
 - Example:

```
string s1, s2, s3;  
...  
s3 = s1 + s2;
```
 - If s3 is not large enough to contain s1 + s2, more space is allocated

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 46

46

string Constructors

- The default string constructor initializes the string to the empty string
- Another string constructor takes a C-string argument
 - Example:

```
string phrase; // empty string
string noun("ants"); // a string
version of "ants"
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 47

47

Mixing strings and C-strings

- It is natural to work with strings in the following manner

```
string phrase = "I love" + adjective
+ " " + noun + "!";
```
- It is not so easy for C++! It must either convert the null-terminated C-strings, such as "I love", to strings, or it must use an overloaded + operator that works with strings and C-strings

Display 8.4

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 48

48

Display 8.4



Program Using the Class string

```
//Demonstrates the standard class string.
#include <iostream>
#include <string>
using namespace std;

int main( )
{
    string phrase;
    string adjective("fried"), noun("ants");
    string wish = "Bon appetite!";

    phrase = "I love " + adjective + " " + noun + "!";
    cout << phrase << endl
          << wish << endl;

    return 0;
}
```

Initialized to the empty string (points to `string phrase;`)

Two ways of initializing a string variable (points to `string adjective("fried"), noun("ants");` and `string wish = "Bon appetite!";`)

Sample Dialogue

I love fried ants!
Bon appetite!

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 49

49

I/O With Class string

- The insertion operator << is used to output objects of type string
 - Example:

```
string s = "Hello Mom!";
cout << s;
```
- The extraction operator >> can be used to input data for objects of type string
 - Example:

```
string s1;
cin >> s1;
```

>> skips whitespace and stops on encountering more whitespace

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 50

50

getline and Type string

- A `getline` function exists to read entire lines into a string variable
 - This version of `getline` is not a member of the `istream` class, it is a non-member function Syntax for using this `getline` is different than that used with `cin`: `cin.getline(...)`
- Syntax for using `getline` with string objects:
`getline(Istream_Object, String_Object);`

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 51

51

getline Example

- This code demonstrates the use of `getline` with string objects

```
string line;  
cout << "Enter a line of input:\n";  
getline(cin, line);  
cout << line << "END OF OUTPUT\n";
```

Output could be:

```
Enter some input:  
Do be do to you!  
Do be do to you!END OF OUTPUT
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 52

52

Character Input With strings

- The extraction operator cannot be used to read a blank character
- To read one character at a time remember to use `cin.get`
 - `cin.get` reads values of type `char`, not type `string`
- The use of `getline`, and `cin.get` for string input are demonstrated in

Display 8.5 (1)

Display 8.5 (2)

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 53

53

DISPLAY 8.5 Program Using the Class string (part 1 of 2)

```
1 //Demonstrates getline and cin.get.
2 #include <iostream>
3 #include <string>
4 void newLine( );
5 int main( )
6 {
7     using namespace std;
8
9     string firstName, lastName, recordName;
10    string motto = "Your records are our records.";
11
12    cout << "Enter your first and last name:\n";
13    cin >> firstName>>lastName;
14    newLine( );
15
16    recordName = lastName + ", " + firstName;
17    cout << "Your name in our records is: ";
18    cout << recordName<<endl;
19
20    cout << "Our motto is\n"
21    << motto <<endl;
22    cout << "Please suggest a better (one-line) motto:\n";
23    getline(cin, motto);
24    cout << "Our new motto will be:\n";
25    cout << motto <<endl;
26
27    return 0;
28 }
29
30 //Uses iostream:
31 void newLine( )
32 {
33     using namespace std;
```

Display 8.5 (1/2)

Back

Next

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 54

54

Display 8.5 (2/2)



```
31     char nextChar;  
32     do  
33     {  
34         cin.get(nextChar);  
35     } while (nextChar != '\n');  
36 }
```

Sample Dialogue

```
Enter your first and last name:  
B'Elanna Torres  
Your name in our records is: Torres, B'Elanna  
Our motto is  
Your records are our records.  
Please suggest a better (one-line) motto:  
Our records go where no records dared to go before.  
Our new motto will be:  
Our records go where no records dared to go before.
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 55

55

Another Version of getline

- The versions of `getline` we have seen, stop reading at the end of line marker `\n`
- `getline` can stop reading at a character specified in the argument list
 - This code stops reading when a `'?'` is read

```
string line;  
cout <<"Enter some input: \n";  
getline(cin, line, '?');
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 56

56

getline Returns a Reference

- `getline` returns a reference to its first argument
- This code will read in a line of text into `s1` and a string of non-whitespace characters into `s2`:

```
string s1, s2;  
getline(cin, s1) >> s2;  
     returns  
cin >> s2;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 57

57

getline Declarations

- These are the declarations of the versions of `getline` for string objects we have seen

```
istream& getline(istream& ins,  
string& strVar, char delimiter);
```

```
istream& getline(istream& ins,  
string& strVar);
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 58

58

Mixing cin >> and getline

- Recall cin >> n skips whitespace to find what it is to read then stops reading when whitespace is found
- cin >> leaves the '\n' character in the input stream

- Example:

```
int n;  
string line;  
cin >> n;  
getline(cin, line);
```

leaves the '\n' which immediately ends getline's reading...line is set equal to the empty string

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 59

59

ignore

- ignore is a member of the istream class
- ignore can be used to read and discard all the characters, including '\n' that remain in a line
 - Ignore takes two arguments
 - First, the maximum number of characters to discard
 - Second, the character that stops reading and discarding
 - Example:

```
cin.ignore(1000, '\n');
```

 reads up to 1000 characters or to '\n'

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 60

60

String Processing

- The string class allows the same operations we used with C-strings...and more
 - Characters in a string object can be accessed as if they are in an array
 - `last_name[i]` provides access to a single character as in an array
 - Index values are not checked for validity!

Display 8.6

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 61

61

DISPLAY 8.6 A string Object Can Behave Like an Array

```
1 //Demonstrates using a string object as if it were an array.
2 #include <iostream>
3 #include <string>
4 using namespace std;
5 int main( )
6 {
7     string firstName, lastName;
8     cout << "Enter your first and last name:\n";
9     cin >> firstName>>lastName;
10    cout << "Your last name is spelled:\n";
11    int i;
12    for (i = 0; i < lastName.length( ); i++)
13    {
14        cout << lastName[i] << " ";
15        lastName[i] = '-';
16    }
17    cout << endl;
18    for (i = 0; i < lastName.length( ); i++)
19        cout << lastName[i] << " "; //Places a "-" under each letter.
20    cout << endl;
21    cout << "Good day " << firstName << endl;
22    return 0;
23 }
```

Sample Dialogue

```
Enter your first and last name:
John Crichton
Your last name is spelled:
C r i c h t o n
-----
Good day John
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 62

62

Display 8.6

Back

Next

Member Function length

- The string class member function length returns the number of characters in the string object:

- Example:

```
int n = stringVar.length( );
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 63

63

Member Function at

- at is an alternative to using []'s to access characters in a string.

- at checks for valid index values

- Example: `string str("Mary");`

Equivalent  `cout << str[6] << endl;`
`cout << str.at(6) << endl;`

Equivalent  `str[2] = 'X';`
`str.at(2) = 'X';`

Other string class functions are found in **Display 8.7**

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 64

64

Member Functions of the Standard Class string

Example	Remarks
Constructors	
<code>string str;</code>	Default constructor creates empty string object <code>str</code> .
<code>string str("sample");</code>	Creates a string object with data "sample".
<code>string str(a_string);</code>	Creates a string object <code>str</code> that is a copy of <code>a_string</code> ; <code>a_string</code> is an object of the class <code>string</code> .
Element access	
<code>str[i]</code>	Returns read/write reference to character in <code>str</code> at index <code>i</code> . Does not check for illegal index.
<code>str.at(i)</code>	Returns read/write reference to character in <code>str</code> at index <code>i</code> . Same as <code>str[i]</code> , but this version checks for illegal index.
<code>str.substr(position, length)</code>	Returns the substring of the calling object starting at <code>position</code> and having <code>length</code> characters.
Assignment/modifiers	
<code>str1 = str2;</code>	Initializes <code>str1</code> to <code>str2</code> 's data.
<code>str1 += str2;</code>	Character data of <code>str2</code> is concatenated to the end of <code>str1</code> .
<code>str.empty()</code>	Returns <code>true</code> if <code>str</code> is an empty string; <code>false</code> otherwise.
<code>str1 + str2</code>	Returns a string that has <code>str2</code> 's data concatenated to the end of <code>str1</code> 's data.
<code>str.insert(pos, str2);</code>	Inserts <code>str2</code> into <code>str</code> beginning at position <code>pos</code> .
<code>str.remove(pos, length);</code>	Removes substring of size <code>length</code> , starting at position <code>pos</code> .
Comparison	
<code>str1 == str2 str1 != str2</code>	Compare for equality or inequality; returns a Boolean value.
<code>str1 < str2 str1 > str2</code>	Four comparisons. All are lexicographical comparisons.
<code>str1 <= str2 str1 >= str2</code>	
Finds	
<code>str.find(str1)</code>	Returns index of the first occurrence of <code>str1</code> in <code>str</code> .
<code>str.find(str1, pos)</code>	Returns index of the first occurrence of string <code>str1</code> in <code>str</code> ; the search starts at position <code>pos</code> .
<code>str.find_first_of(str1, pos)</code>	Returns the index of the first instance in <code>str</code> of any character in <code>str1</code> , starting the search at position <code>pos</code> .
<code>str.find_first_not_of(str1, pos)</code>	Returns the index of the first instance in <code>str</code> of any character not in <code>str1</code> , starting the search at position <code>pos</code> .

Display 8.7

Back
Next

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.
Slide 8- 65

65

string class to numbers

- C++11 has new functions to convert a string class object to a number

```

int i;
double d;
string s;
i = stoi("35"); // Converts the string "35" to an integer 35
d = stod("2.5"); // Converts the string "2.5" to the double 2.5

```

C++11 has new functions to convert a string class object to a number

```

string s = to_string(1.2*2); // "2.4" stored in s

```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.
Slide 1- 66

66

Comparison of strings

- Comparison operators work with string objects
 - Objects are compared using lexicographic order (Alphabetical ordering using the order of symbols in the ASCII character set.)
 - `==` returns true if two string objects contain the same characters in the same order
 - Remember `strcmp` for C-strings?
 - `<`, `>`, `<=`, `>=` can be used to compare string objects

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 67

67

Program Example: Palindrome Testing

- A palindrome is a string that reads the same from front to back as it does from back to front
 - This program ignores spaces and punctuation
 - Upper and lowercase versions of letters are considered the same letter
 - Examples: Able was I 'ere I saw Elba.
Madam, I'm Adam.
A man, a plan, a canal, Panama.
Racecar

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 68

68

Palindrome Testing: removePunct

- `removePunct` removes punctuation from a string
 - `removePunct` compares each character in the string to the characters in a string containing all the punctuation characters and the space character.
 - If a match is not found, the character is added to the string `noPunct`
 - `noPunct`, the original string less any punctuation or spaces, is returned

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 69

69

Palindrome Testing: substr

- The `substr` member function is used to locate a substring within a string
 - `removePunct` uses `substr` to extract a single character at a time from the source string. The character is stored in `aChar`.
 - `removePunct` then uses function `find` to see if the character in `aChar` is in the string of punctuation characters

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 70

70

Palindrome Testing: The Program

- The entire palindrome testing program is found in

Display 8.8 (1)

Display 8.8 (2)

Display 8.8 (3)

Display 8.8 (4)

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 71

71

Display 8.8 (1/4)

← Back

Next →

DISPLAY 8.8 Palindrome Testing Program

```
1 //Test for palindrome property.
2 #include <iostream>
3 #include <string>
4 #include <cctype>
5 using namespace std;
6 void swap (char& v1, char& v2);
7 //Interchanges the values of v1 and v2.
8 string reverse(const string& s);
9 //Returns a copy of s but with characters in reverse order.
10 string removePunct(const string& s, const string& punct);
11 //Returns a copy of s with any occurrences of characters
12 //in the string punct removed.
13 string makeLower(const string& s);
14 //Returns a copy of s that has all uppercase
15 //characters changed to lowercase, other characters unchanged.
16 bool isPal(const string& s);
17 //Returns true if s is a palindrome, false otherwise.
18 int main( )
19 {
20     string str;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 72

72

Display 8.8 (2/4)

Back

Next

```

21     cout << "Enter a candidate for palindrome test\n";
22     << "followed by pressing Return.\n";
23     getline(cin, str);
24     if (isPal(str))
25         cout << "\"" << str << "\" is a palindrome.";
26     else
27         cout << "\"" << str << "\" is not a palindrome.";
28     cout << endl;
29     return 0;
30 }
31
32 void swap(char& v1, char& v2)
33 {
34     char temp = v1;
35     v1 = v2;
36     v2 = temp;
37 }
38
39 string reverse(const string& s)
40 {
41     int start = 0;
42     int end = s.length( );
43     string temp(s);
44
45     while (start < end)
46     {
47         end--;
48         swap(temp[start], temp[end]);
49         start++;
50     }
51     return temp;
52 }

```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 73

73

Display 8.8 (3/4)

Back

Next

```

53 //Uses <ctype> and <string>
54 string makeLower(const string& s)
55 {
56     string temp(s);
57     for (int i = 0; i < s.length( ); i++)
58         temp[i] = tolower(s[i]);
59     return temp;
60 }
61
62 string removePunct(const string& s, const string& punct)
63 {
64     string noPunct; //initialized to empty string
65     int sLength = s.length( );
66     int punctLength = punct.length( );
67     for (int i = 0; i < sLength; i++)
68     {
69         string aChar = s.substr(i,1); //A one-character string
70         int location = punct.find(aChar, 0);
71         //Find location of successive characters
72         //of src in punct.
73         if (location < 0 || location >= punctLength)
74             noPunct = noPunct + aChar; //aChar not in punct, so keep it
75     }
76     return noPunct;
77 }
78
79 //uses functions makeLower, removePunct
80 bool isPal(const string& s)
81 {
82     string punct(".,:;?!\'\" "); //includes a blank
83     string str(s);
84     str = makeLower(str);
85     string lowerStr = removePunct(str, punct);
86     return (lowerStr == reverse(lowerStr));
87 }

```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 74

74

Display 8.8 (4/4)

Back

Next

Palindrome Testing Program (part 4 of 4)

Sample Dialogues

Enter a candidate for palindrome test
followed by pressing Return.
Madam, I'm Adam.
"Madam, I'm Adam." is a palindrome.

Enter a candidate for palindrome test
followed by pressing Return.
Radar
"Radar" is a palindrome.

Enter a candidate for palindrome test
followed by pressing Return.
Am I a palindrome?
"Am I a palindrome?" is not a palindrome.

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 75

75

string Objects to C-strings

- Recall the automatic conversion from C-string to string:

```
char aCString[] = "C-string";  
stringVariable = aCString;
```

- strings are not converted to C-strings
- Both of these statements are illegal:
 - `aCString = stringVariable;`
 - `strcpy(aCString, stringVariable);`

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 76

76

Converting strings to C-strings

- The string class member function `c_str` returns the C-string version of a string object

- Example:

```
strcpy(aCString, stringVariable.c_str( ));
```

- This line is still illegal

```
aCString = stringVariable.c_str( ) ;
```

- Recall that operator `=` does not work with C-strings

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 77

77

Section 8.2 Conclusion

- Can you
 - Show how a string object can be used like a C-string?
 - Write code to read an entire line into a string object?
 - Use the string function `at` to access individual characters in a string object?
 - Write code to convert a string to a C-string?

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 78

78

8.3

Vectors



Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

79

Vectors

- Vectors are like arrays that can change size as your program runs
- Vectors, like arrays, have a base type
- To declare an empty vector with base type int:

```
vector<int> v;
```

 - `<int>` identifies vector as a template class
 - You can use any base type in a template class:

```
vector<string> v;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 80

80

Accessing vector Elements

- Vectors elements are indexed starting with 0
 - []'s are used to read or change the value of an item:

```
v[i] = 42;  
cout << v[i];
```

- []'s cannot be used to initialize a vector element

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 81

81

Initializing vector Elements

- Elements are added to a vector using the member function `push_back`
 - `push_back` adds an element in the next available position

- Example:

```
vector<double> sample;  
sample.push_back(0.0);  
sample.push_back(1.1);  
sample.push_back(2.2);
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 82

82

The `size` of a vector

- The member function `size` returns the number of elements in a vector
- Example: To print each element of a vector given the previous vector initialization:

```
for (int i= 0; i < sample.size( ); i++)  
    cout << sample[i] << endl;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 83

83

The Type `unsigned int`

- The vector class member function `size` returns an **unsigned int**
 - Unsigned int's are nonnegative integers
 - Some compilers will give a warning if the previous for-loop is not changed to:

```
for (unsigned int i= 0; i <  
    sample.size( ); i++)  
    cout << sample[i] << endl;
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 84

84

Alternate vector Initialization

- A vector constructor exists that takes an integer argument and initializes that number of elements

- Example: `vector<int> v(10);`

initializes the first 10 elements to 0

`v.size( )` would return 10

- `[]`'s can now be used to assign elements 0 through 9
- `push_back` is used to add elements beyond the current size of the vector.

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 85

85

Vector Initialization With Classes

- The vector constructor with an integer argument
 - Initializes elements of number types to zero
 - Initializes elements of class types using the default constructor for the class

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 86

86

The vector Library

- To use the vector class
 - Include the vector library

```
#include <vector>
```

- Vector names are placed in the standard namespace so the usual using directive is needed:

```
using namespace std; Display 8.9
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 87

87

Using a Vector

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v;
    cout << "Enter a list of positive numbers.\n"
          << "Place a negative number at the end.\n";

    int next;
    cin >> next;
    while (next > 0)
    {
        v.push_back(next);
        cout << next << " added. ";
        cout << "v.size() = " << v.size() << endl;
        cin >> next;
    }

    cout << "You entered:\n";
    for (unsigned int i = 0; i < v.size(); i++)
        cout << v[i] << " ";
    cout << endl;

    return 0;
}
```

Sample Dialogue

```
Enter a list of positive numbers.
Place a negative number at the end.
2 4 6 8 -1
2 added. v.size() = 1
4 added. v.size() = 2
6 added. v.size() = 3
8 added. v.size() = 4
You entered:
2 4 6 8
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 88

88

Display 8.9

Back

Next

vector Issues

- Attempting to use `[ ]` to set a value beyond the size of a vector may not generate an error
 - The program will probably misbehave
- The assignment operator with vectors does an element by element copy of the right hand vector
 - For class types, the assignment operator must make independent copies

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 89

89

vector Efficiency

- A vector's capacity is the number of elements allocated in memory
 - Accessible using the `capacity( )` member function
- Size is the number of elements initialized
- When a vector runs out of space, the capacity is automatically increased
 - A common scheme is to double the size of a vector
 - More efficient than allocating smaller chunks of memory

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 90

90

Controlling vector Capacity

- When efficiency is an issue
 - Member function `reserve` can increase the capacity of a vector
 - Example:

```
v.reserve(32); // at least 32 elements
```

```
v.reserve(v.size() + 10); // at least 10 more
```
 - `resize` can be used to shrink a vector
 - Example:

```
v.resize(24);
```



```
//elements beyond 24 are lost
```

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 91

91

Section 8.3 Conclusion

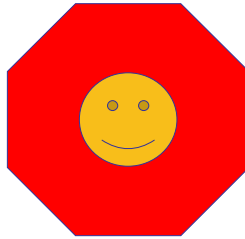
- Can you
 - Declare and initialize a vector of 10 doubles?
 - Write code to increase the size of a vector in at least two different ways?
 - Describe the difference between a vector's size and its capacity?

Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8- 92

92

Chapter 8 -- End



Copyright © 2018 Pearson Addison-Wesley. All rights reserved.

Slide 8-93